

# DevJobs

## Project Documentation

A modern, full-stack job board for developers — built with  
Next.js 14 · Prisma ORM · Tailwind CSS · NextAuth.js

Next.js 14

TypeScript

Prisma

Tailwind  
CSS

GitHub  
Actions

Vercel

|         |              |
|---------|--------------|
| Version | 1.0.0        |
| Date    | 2024         |
| License | MIT          |
| Author  | DevJobs Team |

# Table of Contents

---

1. Overview
2. Features
3. Tech Stack
4. Project Structure
5. Getting Started
6. Database
7. Authentication
8. API Reference
9. CI/CD Pipeline
10. Deployment to Vercel
11. Environment Variables
12. Contributing
13. License

# 1. Overview

---

**DevJobs** is a production-ready job board platform that connects software developers with top tech companies. Employers can post job listings in seconds; developers can search, filter, and apply — all within the same platform.

## Business Value

---

- Employers reach a **targeted audience** of active developers
- Developers find **curated, tech-focused** opportunities
- Featured listings give companies **premium visibility**
- Real-time **application tracking** for both sides

■ DevJobs is designed to be deployed on Vercel with a serverless PostgreSQL database (Neon / Supabase), making it completely free to host at scale.

## 2. Features

| Feature        | Description                                                        |
|----------------|--------------------------------------------------------------------|
| Smart Search   | Full-text search by title, company, location, and skills           |
| Filter by Type | Full-time, Part-time, Contract, Remote, Freelance, Internship      |
| Featured Jobs  | Promoted listings with priority placement and visual badge         |
| Job Posting    | Employers post jobs with rich descriptions, salary, and skill tags |
| Authentication | Email/password + GitHub OAuth via NextAuth.js                      |
| Dashboard      | Job seekers track all applications with status (Pending → Offered) |
| Responsive UI  | Mobile-first design, works on all screen sizes                     |
| SEO-ready      | Open Graph meta tags, semantic HTML throughout                     |
| REST API       | JSON API for programmatic job and application access               |

## 3. Tech Stack

| Layer      | Technology                       | Notes                                             |
|------------|----------------------------------|---------------------------------------------------|
| Framework  | Next.js 14 (App Router)          | Server components, file-based routing, API routes |
| Language   | TypeScript                       | End-to-end type safety                            |
| Styling    | Tailwind CSS                     | Utility-first CSS, custom component classes       |
| Database   | SQLite (dev) / PostgreSQL (prod) | Managed by Prisma ORM                             |
| Auth       | NextAuth.js v4                   | Credentials + GitHub OAuth, JWT sessions          |
| Validation | Zod                              | Schema-based request validation                   |
| Deployment | Vercel                           | Edge network, preview deployments                 |
| CI/CD      | GitHub Actions                   | Lint → Build → Deploy pipeline                    |

## 4. Project Structure

### Directory Layout

```
devjobs/
├── prisma/
│   ├── schema.prisma      # Database schema (User, Job, Application)
│   └── seed.ts            # Seed script - 10 sample tech jobs
├── src/
│   ├── app/
│   │   ├── layout.tsx     # Root layout (Navbar + Footer)
│   │   ├── page.tsx       # Homepage
│   │   └── jobs/
│   │       ├── page.tsx   # Browse all jobs with filters
│   │       └── [id]/page.tsx # Job detail + apply button
│   │       ├── post-job/  # Employer job posting form
│   │       ├── dashboard/ # Applicant dashboard
│   │       ├── auth/      # Sign in + register pages
│   │       └── api/       # REST route handlers
│   ├── components/
│   │   ├── Navbar.tsx     # Sticky nav with auth state
│   │   ├── Footer.tsx    # Site-wide footer
│   │   ├── JobCard.tsx   # Job listing card component
│   │   ├── SearchBar.tsx # Keyword + location + type filter
│   │   ├── ApplyButton.tsx # Apply flow with cover letter
│   │   └── StatsSection.tsx # Platform statistics
│   └── lib/
│       ├── db.ts          # Prisma singleton
│       ├── auth.ts        # NextAuth configuration
│       ├── jobs.ts        # Job query helpers
│       └── utils.ts       # Date, class, label utilities
├── .github/workflows/
│   ├── ci-cd.yml          # Main CI/CD pipeline
│   └── security.yml       # Weekly npm audit
├── vercel.json            # Vercel deploy config
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
└── .env.example
```

## 5. Getting Started

---

### Prerequisites

---

- Node.js  $\geq 18$  (<https://nodejs.org/>)
- npm  $\geq 9$

### Step 1 – Clone the repository

---

```
git clone https://github.com/YOUR_USERNAME/devjobs.git
cd devjobs
```

### Step 2 – Install dependencies

---

```
npm install
```

### Step 3 – Configure environment variables

---

```
cp .env.example .env
```

Edit **.env** and fill in the required values (see Section 11).

### Step 4 – Set up the database

---

```
npm run db:push    # push Prisma schema to SQLite
npm run db:seed    # load 10 sample job listings
```

### Step 5 – Start the development server

---

```
npm run dev
```

Open <http://localhost:3000> in your browser to see the app running.

### Step 6 – Explore the database (optional)

---

```
npm run db:studio  # opens Prisma Studio at localhost:5555
```

## 6. Database

DevJobs uses **Prisma ORM** with **SQLite** for local development and **PostgreSQL** for production. The schema defines four models:

| Model       | Description                                                                        |
|-------------|------------------------------------------------------------------------------------|
| User        | Accounts for job seekers and employers. Stores name, email, hashed password, role. |
| Job         | Job listings with title, company, location, type, salary, description, skill tags. |
| Application | Many-to-many join between User and Job. Tracks cover letter and status.            |
| SavedJob    | Bookmarked jobs per user.                                                          |

### Available npm Scripts

```
npm run db:migrate    # create a new migration after schema changes
npm run db:push       # push schema changes without migration (dev only)
npm run db:seed       # seed the database with sample data
npm run db:studio     # open Prisma Studio GUI
```

### Switching to PostgreSQL (Production)

Edit **prisma/schema.prisma** and change the datasource provider:

```
datasource db {
  provider = "postgresql" # change from "sqlite"
  url      = env("DATABASE_URL")
}
```

Then run:

```
npx prisma migrate deploy
```

## 7. Authentication

Authentication is handled by **NextAuth.js v4** with JWT sessions. Two providers are configured:

| Provider     | Description                                                     |
|--------------|-----------------------------------------------------------------|
| Credentials  | Email + password. Passwords are hashed with bcrypt (12 rounds). |
| GitHub OAuth | Sign in with a GitHub account. Requires a GitHub OAuth App.     |

### Setting up GitHub OAuth

1. Go to GitHub → Settings → Developer settings → OAuth Apps
2. Click **New OAuth App**
3. Set **Homepage URL** to your app URL
4. Set **Callback URL** to **https://YOUR\_DOMAIN/api/auth/callback/github**
5. Copy the Client ID and Secret into your **.env** file

### User Roles

| Role      | Permissions                                       |
|-----------|---------------------------------------------------|
| JOBSEEKER | Browse jobs, apply, view dashboard (default role) |
| EMPLOYER  | Post jobs, manage listings                        |
| ADMIN     | Full platform access                              |



## 8. API Reference

All API routes live under `/api/` and return JSON.

### 8.1 GET `/api/jobs`

Returns a paginated list of active job listings.

| Parameter | Type   | Default | Description                               |
|-----------|--------|---------|-------------------------------------------|
| q         | string | —       | Full-text search keyword                  |
| type      | string | —       | Job type filter (FULL_TIME, REMOTE, etc.) |
| location  | string | —       | Location substring filter                 |
| limit     | number | 20      | Results per page                          |
| skip      | number | 0       | Pagination offset                         |

Example request

```
GET /api/jobs?q=react&type;=REMOTE&limit;=10
```

### 8.2 POST `/api/jobs`

Creates a new job listing.

Request body (JSON)

```
{
  "title":      "Senior React Developer",
  "company":    "Acme Corp",
  "location":   "Remote",
  "type":       "FULL_TIME",
  "salary":     "$120k-$150k",
  "description": "We are looking for...",
  "tags":       ["React", "TypeScript"],
  "url":        "https://acme.com/apply"
}
```

### 8.3 POST `/api/applications` (auth required)

Submit a job application for the authenticated user.

Request body (JSON)

```
{
```

```
"jobId":      "clxxxxxxxx",  
"coverLetter": "I am excited to apply because..."  
}
```

## 8.4 GET /api/applications (auth required)

---

Returns all applications submitted by the currently logged-in user.

## 8.5 POST /api/auth/register

---

Creates a new user account.

Request body (JSON)

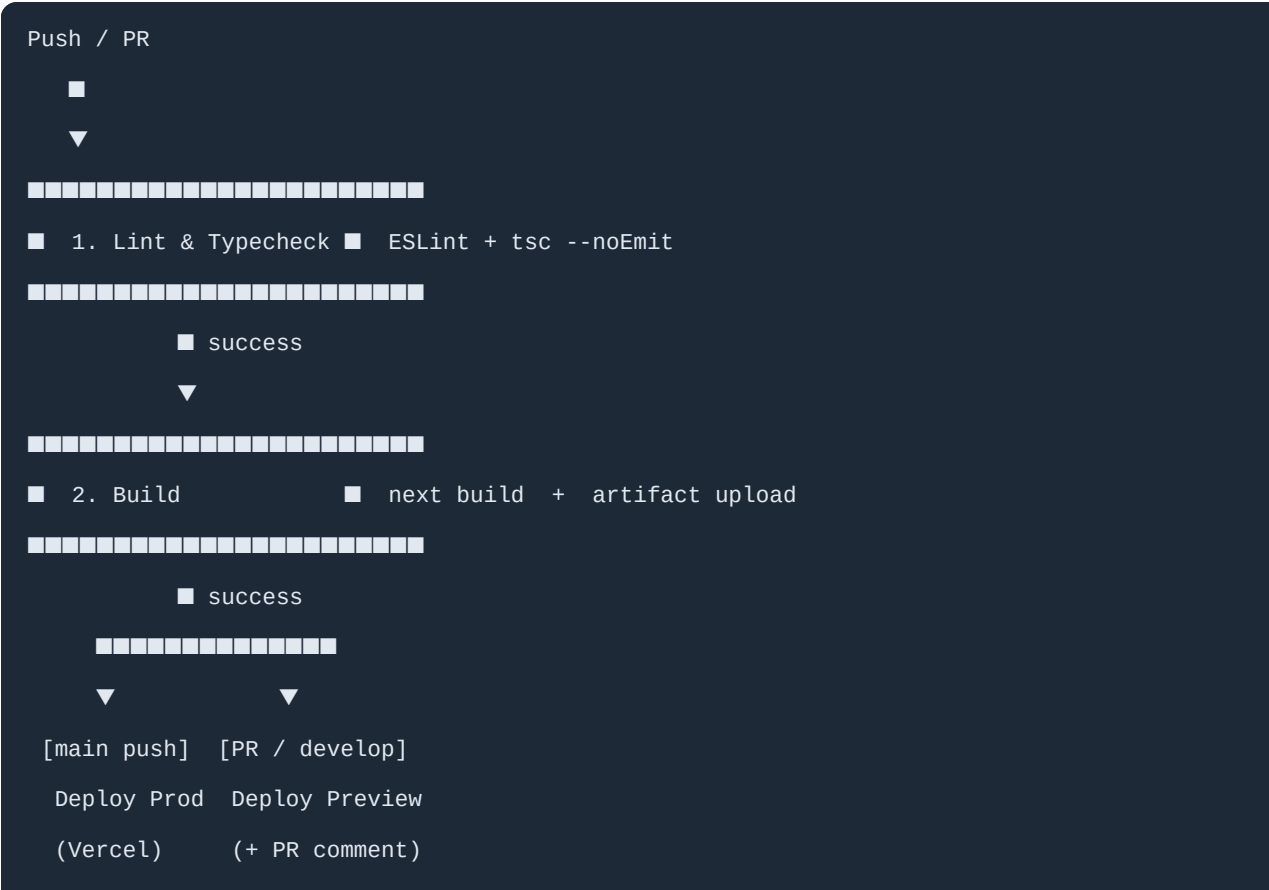
```
{  
  "name":      "Jane Doe",  
  "email":     "jane@example.com",  
  "password":  "securepassword",  
  "role":      "JOBSEEKER"  
}
```

## 9. CI/CD Pipeline

The pipeline is defined in `.github/workflows/ci-cd.yml`. It is triggered on every push to **main** or **develop** and on all Pull Requests targeting **main**.

| Job                  | Trigger              | Description                                            |
|----------------------|----------------------|--------------------------------------------------------|
| 1. lint              | All pushes / PRs     | ESLint + tsc --noEmit                                  |
| 2. build             | After lint passes    | next build + Prisma generate. Uploads .next/ artifact. |
| 3. deploy-production | Push to main only    | vercel deploy --prod via Vercel CLI                    |
| 4. deploy-preview    | PRs + develop branch | vercel deploy (preview URL). Comments URL on the PR.   |

### Pipeline Flow



### Security Workflow

A separate workflow `.github/workflows/security.yml` runs `npm audit --audit-level=high` every Monday at 09:00 UTC to catch newly reported dependency vulnerabilities.

## 10. Deployment to Vercel

### Step 1 – Install Vercel CLI

```
npm install -g vercel
```

### Step 2 – Link the project

```
cd devjobs  
vercel link
```

### Step 3 – Set GitHub Secrets

Navigate to **GitHub repo** → **Settings** → **Secrets and variables** → **Actions** and add the following secrets:

| Secret          | Description                                                                              |
|-----------------|------------------------------------------------------------------------------------------|
| VERCEL_TOKEN    | API token from <a href="https://vercel.com/account/tokens">vercel.com/account/tokens</a> |
| NEXTAUTH_SECRET | Random 32-char string ( <code>openssl rand -base64 32</code> )                           |

### Step 4 – Set Vercel Environment Variables

Navigate to **Vercel project** → **Settings** → **Environment Variables** and add:

| Variable        | Value                                                                                                |
|-----------------|------------------------------------------------------------------------------------------------------|
| DATABASE_URL    | PostgreSQL connection string (Neon / Supabase / PlanetScale)                                         |
| NEXTAUTH_SECRET | Same value as GitHub secret                                                                          |
| NEXTAUTH_URL    | Your Vercel deployment URL, e.g. <a href="https://devjobs.vercel.app">https://devjobs.vercel.app</a> |
| GITHUB_ID       | GitHub OAuth App Client ID (optional)                                                                |
| GITHUB_SECRET   | GitHub OAuth App Client Secret (optional)                                                            |

### Step 5 – Deploy

```
git push origin main # triggers CI/CD pipeline automatically
```

- Every push to main triggers the full CI/CD pipeline which lints, builds, and deploys to Vercel production automatically. Pull requests get a preview URL commented on them.

## Manual Deploy (without CI/CD)

---

```
vercel --prod
```

# 11. Environment Variables

Copy `.env.example` to `.env` and populate all required values before running the app locally or deploying.

| Variable        | Required | Description                                         |
|-----------------|----------|-----------------------------------------------------|
| DATABASE_URL    | Required | SQLite file path (dev) or PostgreSQL URL (prod)     |
| NEXTAUTH_SECRET | Required | Secret key used by NextAuth to sign JWT tokens      |
| NEXTAUTH_URL    | Required | Full URL of the app (http://localhost:3000 for dev) |
| GITHUB_ID       | Optional | GitHub OAuth App Client ID                          |
| GITHUB_SECRET   | Optional | GitHub OAuth App Client Secret                      |

`.env.example`

```
# .env.example
DATABASE_URL="file:./dev.db"
NEXTAUTH_URL="http://localhost:3000"
NEXTAUTH_SECRET="your-secret-here-change-in-production"
GITHUB_ID=""
GITHUB_SECRET=""
```

Tip: Generate a strong NEXTAUTH\_SECRET with: `openssl rand -base64 32`

## 12. Contributing

Contributions are welcome! Follow these steps:

- 1. Fork the repository on GitHub
- 2. Create a feature branch: `git checkout -b feature/your-feature`
- 3. Make your changes and write tests
- 4. Ensure checks pass: `npm run lint && npx tsc --noEmit`
- 5. Commit: `git commit -m "feat: add company profiles page"`
- 6. Push and open a Pull Request against main

### Commit Convention (Conventional Commits)

| Prefix    | Description                              |
|-----------|------------------------------------------|
| feat:     | New feature                              |
| fix:      | Bug fix                                  |
| docs:     | Documentation changes only               |
| style:    | Formatting, whitespace (no logic change) |
| refactor: | Code refactoring without feature change  |
| test:     | Adding or updating tests                 |
| chore:    | Build process or tooling changes         |

## 13. License

DevJobs is released under the **MIT License**. Copyright © 2024 DevJobs.

```
MIT License

Copyright (c) 2024 DevJobs

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
```

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.